Lista Parte-02

Prof. Dr. Gustavo Teodoro Laureano

Sumário

1 Combinador **2**

2 LED **3**

3 Quantas Letras? **4**

4 Um_Dois_Três **5**

5 Zero Vale Zero **6**

6 Ampulheta **7**

7 Criptografia **9**

8 Desenha bordas (++) **10**

9 Desenha losango (+++) **11**

10 Distância entre pontos **13**

11 Matriz de permutação (+++) **14**

12 Matriz de permutação (+++) **16**

13 Matriz xadrez numerada (++) **18**

14 Operações matriciais (++) **19**

15 Ordenação por Data **20**

16 Prefixo de Uma String (++) **21**

17 Procura Caractere **22**

18 Sequência Espelho **23**

19 Aliteração **24**

20 Avance as Letras **25**

21 Cadê o Wally? **26**

22 Cidade Segura **28**

23 Desenha quadrados (+++) **29**

24 Elementos únicos da Matriz (+++) **31**

25 Frações Equivalentes -Versão 2 **32**

26 Frequencia do Maior e Menor **33**

27 Frequência de Letras **34**

28 Imprime matriz em espiral (+++) **35**

29 Limpa *String* (+++) **37**

30 Limpa *String* (+++) **38**

31 Logotipo na TV da vovó **39**

32 Loteria **40**

33 Matriz bissimétrica (+++) **41**

34 Matriz Ordenada +++ **42**

35 Soma e Subtração de Polinômios **43**

36 Potência de matrizes (+++) **45**

37 Sentença Dançante **46**

38 Estiagem **47**

39 Frequência de *Strings* (++++) **48**

40 Números Racionais (++++) **49**

1 Combinador

 (+)

Implemente um programa denominado combinador, que recebe duas strings e deve combiná-las, alternando as letras de cada string, começando com a primeira letra da primeira string, seguido pela primeira letra da segunda string, em seguida pela segunda letra da primeira string, e assim sucessivamente. As letras restantes da cadeia mais longa devem ser adicionadas ao fim da string resultante e retornada.

Entrada

A entrada contém vários casos de teste. A primeira linha contém um inteiro N que indica a quantidade de casos de teste que vem a seguir. Cada caso de teste é composto por uma linha que contém duas cadeias de caracteres. Cada cadeia de caracteres contém entre 1 e 50 caracteres inclusive.

Saída

Combine as duas cadeias de caracteres da entrada como mostrado no exemplo abaixo e exiba a cadeia resultante.

Exemplo

Entrada

`2`
`Tpo oCder`
`aa bb`

Saída
`TopCoder` `abab`

2 LED

 (+)

João quer montar um painel de leds contendo diversos números. Ele não possui muitos leds, e não tem certeza se conseguirá montar o número desejado. Considerando a configuração dos leds dos números abaixo, faça um algoritmo que ajude João a descobrir a quantidade de leds necessário para montar o valor.



Entrada

A entrada contém um inteiro $N, (1 \leq N \leq 1.000)$ correspondente ao número de casos de teste, seguido de N linhas, cada linha contendo um número $(1 \leq V \leq 10^{100})$ correspondente ao valor que João quer montar com os leds.

Saída

Para cada caso de teste, imprima uma linha contendo o número de leds que João precisa para montar o valor desejado, seguido da palavra "leds".

Exemplo

Entrada
3 115380 2819311 23456
Saída
27 leds 29 leds 25 leds

3 Quantas Letras?

 (+)

Tia Magnólia está ensinando as crianças a reconhecerem letras, e entre as letras quais são vogais e quais são consoantes. Ela precisa fazer vários testes com seus alunos. Ela quer que eles leiam várias linhas de um texto e contem em cada linha quantas letras (maiúsculas ou minúsculas), quantas vogais (maiúsculas ou minúsculas) e quantas consoantes (minúsculas ou maiúsculas) existem em cada linha lida. Como Tia Magnólia possui vários textos, ela gostaria de uma forma automatizada de obter essa contagem para gerar um gabarito que permita a ela verificar se as respostas dos alunos estão corretas ou não. Sabendo que você é "FERA" em processamento de strings, ela quer que você faça um programa que gere essas contagens para ela.

Entrada

A entrada contém vários casos de teste. A primeira linha contém um inteiro N que indica a quantidade de casos de teste. Cada caso de teste consiste de uma única linha de texto. A linha pode conter qualquer tipo de caractere (letras e "não letras"). Uma linha pode conter até 10.000 caracteres.

Saída

Para cada caso de teste, imprima três mensagens, cada uma em uma linha diferente. A primeira mensagem deve estar no seguinte formato: "Letras = x". A segunda mensagem deve ser : "Vogais = y" e a última mensagem deve ser: "Consoantes = z". Os valores de x, y e z nas mensagens correspondem aos totais de, respectivamente, letras, vogais e consoantes encontrados em um caso de teste.

Exemplo

Entrada
4 Este e um caso de teste dos varios possiveis Vem ver vovo! O presidente renunciou? #chapeuzinho#Vermelho#
Saída
Letras = 36 Vogais = 17 Consoantes = 19 Letras = 10 Vogais = 4 Consoantes = 6 Letras = 20 Vogais = 10 Consoantes = 10 Letras = 19 Vogais = 8 Consoantes = 11

4 Um_Dois_Três

 (+)

Seu irmão mais novo aprendeu a escrever apenas um, dois e três, em Inglês. Ele escreveu muitas dessas palavras em um papel e a sua tarefa é reconhecê-las. Nota-se que o seu irmão mais novo é apenas uma criança, então ele pode fazer pequenos erros: para cada palavra, pode haver, no máximo, uma letra errada. O comprimento de palavra é sempre correto. É garantido que cada palavra que ele escreveu é em letras minúsculas, e cada palavra que ele escreveu tem uma interpretação única.

Entrada

A primeira linha contém o número de palavras que o seu irmão mais novo escreveu. Cada uma das linhas seguintes contém uma única palavra com todas as letras em minúsculo. As palavras satisfazem as restrições acima: no máximo uma letra poderia estar errada, mas o comprimento da palavra está sempre correto. Haverá, no máximo, 1000 palavras de entrada.

Saída

Para cada caso de teste, imprima o valor numérico da palavra

Exemplo

Entrada
3 owe too theee
Saída
1 2 3

5 Zero Vale Zero

 (+)

Um dia o Prof. Humberto José Roberto fez o seguinte questionamento: Se o zero a esquerda de um número não tem valor algum, por que teria em outras posições de um número? Analisando da seguinte forma, ele pede sua ajuda para, ao somar dois valores inteiros, que o resultado seja exibido segundo o raciocínio dele, ou seja, sem os Zeros. Por exemplo, ao somar 15 + 5, o resultado seria 20, mas com esta nova ideia, o novo resultado seria 2, e, ao somar 99 + 6, o resultado seria 105, mas com esta nova ideia, o novo resultado seria 15.

Escreva um programa que, dado dois números inteiros, sem o algarismo zero, some os mesmos e, caso o resultado tenha algum algarismo zero, que os retire antes de exibir.

Entrada

Haverá diversos casos de teste. Cada caso de teste inicia com dois inteiros M e N $(1 \leq M \leq N \leq 999.999.999)$. O último caso de teste é indicado quando $N = M = 0$, sendo que este caso não deve ser processado.

Saída

Para cada caso de teste, imprima o resultado da soma dos dois valores, sem os zeros.

Sugestão

Ao somar os dois números utilize a função sprintf() para armazenar a soma em uma string.

Exemplo

Entrada
7 8 15 5 99 6 0 0
Saída
15 2 15

6 Ampulheta

 (++)

O objetivo desse execício é identificar um conjunto de elementos cuja somatória seja o maior valor entre todos os conjuntos definidos por um padrão na forma de ampulheta, em uma matriz de inteiros, quadrada, e de ordem 6. Cada elemento da matriz está no intervalo [-9, 9]. Uma "ampulheta" é formada pelos valores que estão posicionados de acordo com a seguinte configuração: $\begin{bmatrix} a & b & c \\ & d & \\ e & f & g \end{bmatrix}$.

O valor de uma ampulheta é a soma de todos os valores presentes nas respectivas posições. Seu programa deve retornar o maior valor entre todos os valores de ampulheta possíveis na matriz.

Entrada

Uma matriz quadrada de ordem 6.

Saída

Um único inteiro que corresponde à maior soma de todos os valores de ampulheta. Após imprimir o valor quebre a linha.

Exemplo

Entrada	Saída
1 1 1 0 0 0 0 1 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	7

Entrada	Saída
1 1 1 0 0 0 0 1 0 0 0 0 1 1 1 0 0 0 0 0 2 4 4 0 0 0 0 2 0 0 0 0 1 2 4 0	19

Entrada	Saída
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 1	0

Entrada	Saída
5 5 -1 -4 1 6 -2 -3 3 -2 3 -7 9 -4 -5 3 8 -9 -2 -2 7 3 5 -9 -8 -7 6 -3 -2 8 1 9 7 0 3 -2	30

Entrada	Saída
-9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9	-63

Entrada	Saída
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9	63

7 Criptografia

 (++)

Solicitaram para que você construisse um programa simples de criptografia. Este programa deve possibilitar enviar mensagens codificadas sem que alguém consiga lê-las. O processo é muito simples. São feitas três passadas em todo o texto.

Na primeira passada, somente caracteres que sejam letras minúsculas e maiúsculas devem ser deslocadas 3 posições para a direita, segundo a tabela ASCII: letra 'a' deve virar letra 'd', letra 'y' deve virar caractere '|' e assim sucessivamente. Na segunda passada, a linha deverá ser invertida. Na terceira e última passada, todo e qualquer caractere a partir da metade em diante (truncada) devem ser deslocados uma posição para a esquerda na tabela ASCII. Neste caso, 'b' vira 'a' e 'a' vira '`'.

Por exemplo, se a entrada for "Texto #3", o primeiro processamento sobre esta entrada deverá produzir "Wh{wr #3". O resultado do segundo processamento inverte os caracteres e produz "3# rw{hW". Por último, com o deslocamento dos caracteres da metade em diante, o resultado final deve ser "3# rvzgV".

Entrada

A entrada contém vários casos de teste. A primeira linha de cada caso de teste contém um inteiro $N(1 \leq N \leq 10^4)$, indicando a quantidade de linhas que o problema deve tratar. As N linhas contém cada uma delas $M(1 \leq M \leq 10^3)$ caracteres.

Saída

Para cada entrada, deve-se apresentar a mensagem criptografada.

Exemplo

Entrada
4 Texto #3 abcABC1 vxpdylY .ph vv.xwfxo.fd
Saída
3# rvzgV 1FECedc ks. \n{frzx gi.r{hyz-xx

8 Desenha bordas (++)

 (++)

Faça um programa que gere uma matriz de zeros, de tamanho definido pelo usuário, de no máximo 100×100, com uma borda de largura k de valor x.

Entrada

O programa deve ler quatro números inteiros, os dois primeiros relacionados à largura e altura da matriz, o terceio a largura da borda e o por último o valor da borda.

Saída

O programa deve apresentar a matriz como uma imagem PGM, ou seja, seguindo a sequência:

```
P2
largura altura
255
<dados da matriz>
```

Os dados da matriz devem ser impressos sempre com um espaço à direita e seguido de quebra de linha ao final de cada linha da matriz.

Observações

Para testar seu código, você pode redirecionar a saída padrão do seu programa para um arquivo com extensão ".pgm", usando o comando "./programa > img.pgm".

Exemplo

Entrada	Saída
5 5 1 2	P2 5 5 255 2 2 2 2 2 2 0 0 0 2 2 0 0 0 2 2 0 0 0 2 2 2 2 2 2

9 Desenha losango (+++)

 (+++)

Faça um programa que gere uma matriz quadrada de caracteres, de tamanho definido pelo usuário, de no máximo 99×99, com um losango desenhado em seu interior, equivalente ao maior quadrado rotacionado 45 graus. O programa deve ler a dimensão da matrix, o caractere a ser usado para representar a borda do losango e o caractere a ser usado como interior do losango. O restante da matriz deve ser preenchido com o caractere espaço.

Entrada

O programa deve um número inteiro n, ímpar, que representa a dimensão da matriz. Em seguida, deve-se ler dois caracteres, o primeiro representando o caractere de borda e o segundo o caractere de fundo. Para realizar a leitura use a expressão `"%d %c %c"`.

Saída

Caso o valor informado para a dimensão da matriz seja inválido, o programa deve imprimir a mensagem "Dimensao invalida!"e encerrar. Caso contrário, o programa deve imprimir a matriz $n \times n$ gerada, com cada elemento separado por um espaço à direita, (usando `"%c "`) e cada linha separada por uma quebra de linha '\n'.

Exemplo

Entrada 1

```
3 o y
```

Saída 1

```
  o
o y o
  o
```

Entrada 2

```
11 x o
```

Saída 2



10 Distância entre pontos

 (++)

Faça um programa que leia vários pares de pontos no espaço de quatro dimensões e calcule a distância entre eles. Considere que a distância entre dois pontos no espaço tetradimensional $A(a_u, a_x, a_y, a_z)$ e $B(b_u, b_x, b_y, b_z)$ é calculada como:

$$d(A,B) = |BA| = \sqrt{(a_u - b_u)^2 + (a_x - b_x)^2 + (a_y - b_y)^2 + (a_z - b_z)^2} \quad (1)$$

Voce deve usar uma struct para representar cada ponto.

Entrada

A entrada consiste de várias linhas. A primeira linha apresenta um número de pontos N, com $2 \leq N \leq 1000$. As N linhas seguintes apresentam pontos no espaço na forma u,x,y,z, com u,x,y e z números reais tais que $-1000 \leq x,y,z \leq 1000$. Faça um programa que calcule a distância entre dois pontos consecutivos nesta lista. Note que, com exceção do primeiro e último valor de entrada, todos os pontos serão utilizados duas vezes, uma para o cálculo de distância com o ponto que veio antes na lista e outra para o ponto que veio depois.

Saída

A saída consiste de $(N-1)$ linhas, cada uma contendo a distância entre os pontos com 2 casas decimais após a vírgula. Após a impressão do valor de uma distância, o programa deve imprimir o caractere de quebra de linha.

Exemplo

Entrada
2 4 1 0 1 -1 2 1 2
Saída
5.29

Entrada
4 1 1 5 1 2 -1 3 0 4 2 -1 2 -3 4 2 2
Saída
3.16 5.74 7.87

Entrada
4 15.89 0.7 0.53 0.33 0.45 0.38 0.22 0.11 0 0 0 1 0 0 1 0.5
Saída
15.45 1.09 1.12

11 Matriz de permutação (+++)

 (+++)

Dizemos que uma matriz inteira $\mathbf{A}_{n\times n}$ é uma matriz de permutação se em cada linha e em cada coluna houver $n-1$ elementos nulos e um único elemento igual a 1.

A matriz A abaixo é de permutação:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A matriz B abaixo não é de permutação.

$$B = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Dada uma matriz inteira $\mathbf{A}_{n\times n}$, verificar se A é de permutação.

Você deve implementar uma função que recebe a matriz e retorne 0 ou 1, sendo que o 0 (zero) indica que a matriz não é de permutação:

```
/**
 * Função que verifica se a matriz é de permutação
 * @param matriz Indica a matriz a ser verificada
 * @param n indica a dimensão da matriz
 * @param *soma parâmetro de saída, que armazenará a soma de todos os 'n' elementos
   da matriz.
 * @return int
 */
int ehPermutacao( int matriz[500][500], int n, int *soma );
```

Entrada

Na primeira linha ha um inteiro n, 1<n<=500, representando a ordem da matriz quadrada. A seguir haverá n linhas com n inteiros em cada linha separados por um espaço em branco cada, representando os elementos da matriz quadrada.

Saída

Deverá imprimir 3 (três) linhas:

- A dimensão da matriz (n)
- A mensagem "PERMUTACAO"ou "NAO EH PERMUTACAO", que representa se esta **é** ou **não** uma matriz de permutação
- Soma de todos os elementos da matriz

Exemplo

Entrada	Saída
4 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 1	4 PERMUTACAO 4

Entrada	Saída
3 2 -1 0 -1 2 0 0 0 1	3 NAO EH PERMUTACAO 3

12 Matriz de permutação (+++)

 (+++)

Dizemos que uma matriz inteira $\mathbf{A}_{n\times n}$ é uma matriz de permutação se em cada linha e em cada coluna houver $n-1$ elementos nulos e um único elemento igual a 1.

A matriz A abaixo é de permutação:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

A matriz B abaixo não é de permutação.

$$B = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

Dada uma matriz inteira $\mathbf{A}_{n\times n}$, verificar se A é de permutação.

Você deve implementar três funções, uma para criar uma matriz alocada dinamicamente, outra para liberar a memória da matriz e uma função que recebe a matriz e retorne 0 ou 1, sendo que o 0 (zero) indica que a matriz não é de permutação. Abaixo estão os protótipos das funções que devem ser implementadas.

```c
/**
 * Essa função recebe um valor "n" e retorna um ponteiro
 * para uma matriz quadrada de inteiros com dimensão nxn.
 * A memória é alocada dinamicamente com uma estrutura
 * bidimensional.
 *
 * @param n Dimensão da matriz
 * @return int** Ponteiro para a matriz
 */
int ** cria_matrix( int n );

/**
 * Essa função recebe um ponteiro para uma matriz alocada
 * dinamicamente e libera a memória ocupada por ela.
 *
 * @param m Ponteiro para a matriz
 */
void destroi_matrix( int ** m );

/**
 * Essa função recebe um ponteiro para uma matriz alocada
 * dinamicamente, sua dimensão n e retorna 1 caso essa
 * matriz seja uma matriz de permutação ou 0 caso contrário.
 * Além disso, essa função também calcula a soma de todos os
 * elementos da matriz e a retorna via referência pelo
 * ponteiro "soma".
 *
 * @param m Ponteiro para a matriz
 * @param n Dimensão da matriz
 * @param soma Ponteiro para inteiro que retornará a soma dos
 * elementos da matriz
 * @return int 1 caso a matriz seja de permutação
 * e 0 caso contrário
 */
int ePermutacao( int ** m, int n, int * soma );
```

Entrada

Na primeira linha ha um inteiro *n*, representando a ordem da matriz quadrada. A seguir haverá *n* linhas com *n* inteiros em cada linha separados por um espaço em branco cada, representando os elementos da matriz quadrada.

Saída

Deverá imprimir 3 (três) linhas:

- A dimensão da matriz (*n*)
- A mensagem "PERMUTACAO"ou "NAO E PERMUTACAO", que representa se esta **é** ou **não** uma matriz de permutação
- Soma de todos os elementos da matriz

Exemplo

Entrada	Saída
4 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 1	4 PERMUTACAO 4

Entrada	Saída
3 2 -1 0 -1 2 0 0 0 1	3 NAO E PERMUTACAO 3

13 Matriz xadrez numerada (++)

 (++)

Uma matriz xadrez numerada é uma matriz (quadrada ou retangular) composta de números inteiros. O padrão reticulado inicia em 0 (preto) e é alternado com um número sequencial maior a zero (branco).

Entrada

Dois números inteiros que representam o número de linhas e colunas da matriz xadrez.

Saída

Uma matriz xadrez de *m* linhas e *n* colunas.

Exemplo

Entrada	Saída
5 5	0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9 0 10 0 11 0 12 0

Entrada	Saída
3 6	0 1 0 2 0 3 4 0 5 0 6 0 0 7 0 8 0 9

14 Operações matriciais (++)

 (++)

Faça um programa que, dada uma matriz quadrada A de dimensões N x N realize a seguinte operação:

$$\mathrm{tr}(\mathbf{A}) \cdot \mathbf{A} + \mathbf{A}^{\mathsf{T}} \tag{4}$$

Onde:

- $\mathbf{A}$ é a matriz de entrada;
- $\mathbf{A}^{\mathsf{T}}$ é a matriz transposta de $\mathbf{A}$;
- tr é o traço da matriz, ou seja, a soma dos elementos na diagonal principal;

Entrada

A entrada contém apenas um caso de teste. A primeira linha há um inteiro N, 1 < N <= 1000, representando a dimensão das matrizes. A seguir haverão N linhas com N inteiros em cada linha separados por um espaço em branco cada, representando os elementos da matriz A.

Saída

A saída consiste de N linhas com N inteiros em cada linha separados por um espaço em branco cada, representando o resultado da Equação 4.

Exemplo

Entrada	Saída
3 1 2 3 4 5 6 7 8 9	16 34 52 62 80 98 108 126 144

Entrada	Saída
2 34 23 56 98	4522 3092 7415 13034

15 Ordenação por Data

 (++)

Uma determinada professora quer ordenar seus alunos em ordem crescente de idade. Escreva um programa em C que leia os dados dos alunos, entre eles a data de nascimento, ordene os alunos em ordem crescente de idade. Para isso seu programa deve ter uma função *ComparaDataNasc()* que recebe dois parâmetros. O primeiro corresponde a uma struct (ou um ponteiro para uma struct) que contém o dia, o mês e o ano de nascimento de uma aluno. O segundo parâmetro tem o memo tipo de dado do primeiro e contém a data de nascimento do segundo aluno. Essa função retorna 1 se o primeiro aluno é mais novo ou tem a mesma idade do segundo aluno e retorna zero em caso contrário. Essa função deve ser chamada pela função que ordena os alunos em ordem crescente de idade.

Entrada

A entrada contém apenas um caso de teste. A primeira linha da entrada contém um número inteiro n, ($1 \leq n \leq 30$) que corresponde ao número de alunos da turma. Em seguida há n linhas, contendo cada uma:

- a matrícula de um aluno (int);
- o dia de nascimento de um aluno (int);
- o mês de nascimento de um aluno (int);
- o ano de nascimento de um aluno (int);
- o nome de um aluno (no máximo 200 caracteres);

Saída

A saída é formada por n linhas, cada uma correspondendo ao um aluno, ordenadas em ordem crescente de idade dos alunos. Cada linha deve ter o seguinte formato: Matric.: *m* Nome: *n* Data Nasc.: *dd*/*mm*/*aa*, onde *m* é a matricula de um aluno, *n*, o seu nome, e *dd*, *mm* e *aa*, são respectivamente o dia, o mês e o ano do seu nascimento.

Exemplo

Entrada
5 12345 12 07 1978 Felizbina Freitas 23489 11 03 2009 Joao Feliz da Tristeza 98762 05 12 1976 Maria Batista de Souza 34561 11 07 1978 Roberto de Assis 34599 07 05 1976 Luiz Alberto Ferreira
Saída
Matric.: 23489 Nome: Joao Feliz da Tristeza Data Nasc: 11/3/2009 Matric.: 12345 Nome: Felizbina Freitas Data Nasc: 12/7/1978 Matric.: 34561 Nome: Roberto de Assis Data Nasc: 11/7/1978 Matric.: 98762 Nome: Maria Batista de Souza Data Nasc: 5/12/1976 Matric.: 34599 Nome: Luiz Alberto Ferreira Data Nasc: 7/5/1976

16 Prefixo de Uma String (++)

 (++)

Escreva um programa para ler várias linhas na entrada. Cada linha contém um número inteiro seguido por um espaço e por uma string. Para cada linha, o programa deve chamar uma função do tipo **ponteiro para char** que que receba como primeiro parâmetro n o inteiro lido e como segundo parâmetro s a string lida. A função deve alocar espaço suficiente para armazenar os n primeiros caracteres de s (prefixo de s). Deve copiar os n primeiros caracteres de s para essa nova string e retornar o endereço da string criada. Se n for maior que o tamanho da string s, o prefixo corresponde a uma cópia da string s. A função deve retornar NULL, se não conseguir alocar o espaço necessário para um prefixo. Após chamar a função, o programa deve verificar se função retornou um endereço válido de prefixo, e nesse caso, deve imprimir o prefixo e deve liberar a área ocupada pelo prefixo, antes de processar uma nova linha

Entrada

A primeira linha da entrada contém um inteiro positivo $N(1 \leq N \leq 20)$, o qual corresponde ao número de casos de teste. Cada caso de teste corresponde a uma linha. Cada linha possui um número inteiro positivo n, um espaço e uma string s, com no máximo 499 caracteres.

Saída

Para cada caso de teste o programa deve imprimir uma linha contendo o prefixo de tamanho n da string s lida naquele caso de teste.

Exemplo

Entrada

`5`
`1 Universidade Federal de Goias`
`0 Introducao a Programacao`
`3 Universidade Federal de Goias`
`20 Universidade Federal de Goias`
`30 Universidade Federal de Goias`

Saída

`U`
 	
`Uni`
`Universidade Federal`
`Universidade Federal de Goias`

17 Procura Caractere

Escreva um programa para ler várias linhas na entrada. Cada linha contém um caractere seguido por um espaço e por uma string. Para cada linha, o programa deve chamar uma função do tipo int que receba como primeiro parâmetro o caractere lido e como segundo parâmetro a string lida. A função deve retorna o índice do vetor onde o caractere aparece pela primeira vez na string. Se o caractere não aparece na string, a função deve retornar -1.

Entrada

A primeira linha da entrada contém um inteiro positivo $N(1 \leq N \leq 20)$, o qual corresponde ao número de casos de teste. Cada caso de teste corresponde a uma linha. Cada linha possui um caractere, um espaço e uma string, com no máximo 499 caracteres.

Saída

Para cada caso de teste o programa deve imprimir uma das seguintes frases:

- "Caractere *c* encontrado no indice *i* da string.", ou
- "Caractere *c* nao encontrado."

Exemplo

Entrada:
4 o Introducao a Programacao G Universidade Federal de Goias ; Universidade Federal de Goias   Universidade Federal de Goias

Saída:
Caractere o encontrado no indice 4 da string. Caractere G encontrado no indice 24 da string. Caractere ; nao encontrado. Caractere encontrado no indice 12 da string.

Observação: Na última linha de entrada do exemplo, o caractere a ser procurado é o caractere espaço.

18 Sequência Espelho

 (++)

Imprimir números em sequência é uma tarefa relativamente simples. Mas, e quando se trata de uma sequência espelho? Trata-se de uma sequência que possui um número de início e um número de fim, e todos os números entre estes, inclusive estes, são dispostos em uma sequência crescente, sem espaços e, em seguida, esta sequência é projetada de forma invertida, como um reflexo no espelho. Por exemplo, se a sequência for de 7 a 12, o resultado ficaria 789101112211101987.

Entrada

A entrada possui um valor inteiro C indicando a quantidade de casos de teste. Em seguida, cada caso apresenta dois valores inteiros, B e E $(1 \leq B \leq E \leq 12221)$, indicando o início e o fim da sequência.

Saída

Para cada caso de teste, imprima a sequência espelho correspondente.

Sugestão

Utiliza a função sprintf() para imprimir um número inteiro em uma string. Use a função strlen() para obter o tamanho de uma string.

Exemplo

Entrada
3 1 5 10 13 98 101
Saída
1234554321 1011121331211101 98991001011010019989

19 Aliteração

 (+++)

Uma aliteração ocorre quando duas ou mais palavras consecutivas de um texto possuem a mesma letra inicial (ignorando maiúsculas e minúsculas). Sua tarefa é desenvolver um programa que identifique, a partir de uma sequência de palavras, o número de aliterações que essa sequência possui.

Entrada

A entrada contém diversos casos de testes. Cada caso é expresso como um texto em uma única linha, contendo de 1 a 100 palavras separadas por um único espaço, cada palavra tendo de 1 a 50 letras minúsculas ou maiúsculas ('A'-'Z','a'-'z'). A entrada termina em EOF.

Saída

Para cada caso de teste imprima o número de aliterações existentes no texto informado, conforme exemplos abaixo.

Exemplo

Entrada

`He has four fanatic fantastic fans`
`There may be no alliteration in a sequence`
`Round the rugged rock the ragged rascal ran`
`area artic Soul Silly subway ant artic none`

Saída

`2`
`0`
`2`
`3`

20 Avance as Letras

 (+++)

São dadas na entrada uma string A e outra B. Em uma operação você pode escolher uma letra da primeira string e avançar esta letra. Avançar uma letra significa transformá-la na próxima letra do alfabeto, veja que a próxima letra depois de z vem a letra a novamente!

Por exemplo, podemos transformar a string **ab** em **bd** em no mínimo 3 operações: **ab** $\rightarrow$ **bb** $\rightarrow$ **bc** $\rightarrow$ **bd**. Podemos aplicar operações nas letras em qualquer ordem, outra possibilidade seria: **ab** $\rightarrow$ **ac** $\rightarrow$ **bc** $\rightarrow$ **bd**.

Dadas as duas strings, calcule o mínimo número de operações necessárias para transformar a primeira na segunda.

Entrada

Na primeira linha terá um inteiro $T(T \leq 100)$ indicando o número de casos de teste. Para cada caso, na única linha teremos as duas strings $A(1 \leq |A| \leq 10^4$ - sendo que $|A|$ significa o tamanho da string $A)$ e $B(|B| = |A|)$ separadas por um espaço. Ambas as strings são compostas apenas por letras minúsculas do alfabeto e são do mesmo tamanho.

Saída

Para cada caso imprima o número mínimo de operações.

Exemplo

Entrada
3 ab bd abc abc abcdefghiz aaaaaaaaaa
Saída
3 0 173

21 Cadê o Wally?

 (+++)

Wally costuma morar em um ambiente representado por uma matriz bidimensional de números inteiros de ordem n x m (n linhas e m colunas). Ele é único no ambiente e é representado na matriz por quatro números, distribuidos da seguinte forma:

	4	
0	1111	0
	8	

O número 1111 representa a camisa listrada em vermelho e branco, e o número 4 representa seu gorro das mesmas cores. Os números zero e oito representam suas extremidades. A matriz representa um ambiente bidimensional circular:

- Para o índice i=0, a celula à esquerda está no índice m-1.
- Para o índice i=m-1, a celula à direita está no índice 0.
- Para o índice j=0, a celula superior está no índice n-1.
- Para o índice j=n-1, a celula inferior está no índice 0.

Crie um programa que permita imprimir os índices i,j (da matriz) onde está a camisa do Wally. Caso o Wally não estiver na matriz mostre a seguinte mensagem: "WALLY NAO ESTA NA MATRIZ". Na seguinte matriz de 7 linhas e 6 colunas, a camisa do Wally está nos índices i=3, j=0.

0	1111	0	1	0	0
0	0	0	0	1111	0
4	0	1	3	45	53
1111	0	89	211	87	0
8	4	56	4	55	98
0	222	0	11	0	5
0	8	23	8	66	3

Entrada

Dois valores inteiros, n e m, seguidos dos elementos inteiros da matriz de ordem n x m, com n>=3 e m>=3.

Saída

Se o Wally estiver na matriz: o índice i,j da localização do Wally. Caso contrário "WALLY NAO ESTA NA MATRIZ" (sem acentos)

Exemplo

Entrada	Saída
3 7 4 7 7 7 7 7 7 1111 0 7 7 7 7 0 8 7 7 7 7 7 7	1 0

Entrada	Saída
5 4 5 5 8 5 5 5 5 5 5 5 5 5 5 5 4 5 5 0 1111 0	4 2

Entrada	Saída
3 3 1 4 1 0 1111 0 1 8 1	1 1

Entrada	Saída
3 3 0 0 0 0 0 0 0 0 0	WALLY NAO ESTA NA MATRIZ

22 Cidade Segura

 (+++)

Com o aumento da violencia na cidade o prefeito decidiu instalar câmeras de vigilância em todas as esquinas. A cada mês, um mapa atualizado com as câmeras em funcionamento é disponibilizado no site da prefeitura. A secretaria de segurança considera que uma esquina é segura se existem câmeras em funcionamento, pelo menos, duas de suas quatro esquinas. Nesta cidade todas as quadras são quadrados de mesmo tamanho. Sua tarefa é, dado o mapa das câmeras em funcionamento nas esquinas, indicar o status de todas as quadras da cidade.

Entrada

A primeira linha da entrada tem um inteiro positivo N (1 N 100). Nas próximas N+1 linhas, existem N+1 números, que indicam, para cada esquina, a presença de uma câmera em funcionamento ou de uma câmera defeituosa. O número 1 indica que existe uma câmera funcionando na esquina, enquanto o número zero indica que não há câmera funcionando.

Saída

A saída é dada em N linhas. Cada linha tem N caracteres, indicando se a quadra correspondente é segura ou insegura. Se uma quadra é segura, mostre o caractere S, caso contrário mostre o caractere U. Após a última linha não se esqueça de saltar uma linha.

Exemplo

Entrada	Saída
1 1 0 0 0	U

Entrada	Saída
2 1 0 0 1 1 0 0 0 1	SU SS

Entrada	Saída
3 1 1 0 1 1 0 1 0 1 0 0 1 0 1 1 0	SSS SUS SSS

23 Desenha quadrados (+++)

 (+++)

Faça um programa que gere uma imagem PGM com quadrados definidos pelo usuário. A imagem deve ter o fundo preenchido por uma cor, $C \in \{0,255\}$, definida pelo usuário e ter dimensão de no máximo 200×200 *pixels*. O programa deve ler as coordenadas do ponto central do quadrado, a cor (0,255), o raio e o estilo (1 para borda, 2 para preenchido). O programa encerra quando o as coordenadas do quadrado são $x = -1$ e $y = -1$. Os quadrados que excedem as dimensões da imagem não devem ser desenhados.

Entrada

O programa deve ler o tamanho da imagem, um valor para a cor de fundo $C \in \{0,255\}$ e uma sequência de linhas, cada uma composta de números que definem a posição x e y do quadrado, o raio r e o estilo de desenho, sendo 1 para somente borda e 2 para preenchido. Por exemplo, a linha "10 20 8 4 2"é a instrução para desenhar um quadrado preenchido com a cor 8 na linha 20, coluna 10, aresta de tamanho $4 \times 2 + 1$, sendo 4 pontos à esquerda mais 4 prontos à direita do centro do quadrado mais o ponto central. A Figura 1 mostra um exemplo de saída visualizada como uma imagem PGM.

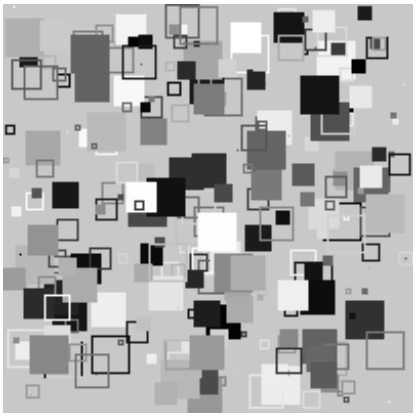


Figura 1: Exemplo de resultado para uma imagem 200x200, cor de fundo 8 e 300 quadrados gerados aleatoriamente.

Saída

O programa deve apresentar a matriz como uma imagem PGM, ou seja, seguindo a sequência:

```
P2
200 200
255
<dados da matriz>
```

Os dados da matriz devem ser impressos sempre com um espaço à direita e seguido de quebra de linha ao final de cada linha da matriz.

Observações

Para testar seu código, você pode redirecionar a saída padrão do seu programa para um arquivo com extensão ".pgm", usando o comando "./programa > img.pgm".

Exemplo

Entrada	Saída
10 0 5 5 4 10 2 1 1 4 1 1 5 5 8 2 2 -1 -1	P2 10 10 255 4 4 4 0 0 0 0 0 0 0 4 0 4 0 0 0 0 0 0 0 4 4 4 0 0 0 0 0 0 0 0 0 0 8 8 8 8 8 0 0 0 0 0 8 8 8 8 8 0 0 0 0 0 8 8 8 8 8 0 0 0 0 0 8 8 8 8 8 0 0 0 0 0 8 8 8 8 8 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

24 Elementos únicos da Matriz (+++)

 (+++)

Dada uma matriz $nl \times nc$, encontre todos os elementos que não se repetem. Considere a dimensão máxima da matriz igual a 10×10.

Entrada

O programa deve ler a quantidade de linhas (nl) e colunas (nc) da matriz e, em seguida, os $nl * nc$ elementos da matriz.

Saída

Caso os números nl ou nc estejam fora do intervalo, o programa imprime a mensagem "dimensao invalida"e encerra. Caso não exista nenhum elemento único, o programa deve imprimir a mensagem "sem elementos unicos". Caso contrário, o programa imprime uma linha contendo os elementos que não se repetem na matriz separados por vírgula e obedecendo a sequencia em que aparecem na matriz (da esquerda para direita e de cima para baixo).

Exemplo

Entrada	Saída
3 3 1 9 3 2 5 4 3 2 1	9,5,4

Entrada	Saída
3 2 2 3 1 2 3 1	sem elementos unicos

Entrada	Saída
0 8	dimensao invalida

25 Frações Equivalentes -Versão 2

 (+++)

Tia Zuleika está ensinado seus alunos sobre frações equivalentes. Ela vai gerar uma lista de frações e vai pedir aos seus alunos para indicarem quais pares de frações são equivalentes entre si, percorrendo a lista da esquerda para direita. Ela pretende gerar uma lista grande, para que todos os alunos da turma tenham oportunidade de responder. Tia Zuleika está muito ocupada e sabendo que você é "fera" em programação, está pedindo sua ajuda para que faça um programa capaz de ler sequências de frações e para cada sequência indicar quais pares são equivalentes entre si. O programa deve comparar a primeira fração com todas as outras seguintes, depois deve comparar a segunda com todas as outras seguintes e assim por diante, de modo que a posição na lista da primeira fração de um par de frações seja sempre menor que a posição da segunda fração do par.

Entrada

A primeira linha da entrada contém o número $n \leq 30$ de casos de teste. Para cada caso de teste há Uma linha na entrada que contém a lista de frações separadas entre si por um espaço. Cada fração tem o seguinte formato: *x*/*y*.

IMPORTANTE: esta versão se difere da primeira versão do problema em relação à entrada de dados. Na primeira versão, cada caso de teste era composto por duas linhas. A primeira continha o tamanho da lista de frações e a segunda linha continha a lista de frações no tamanho especificado na linha anterior. Nessa versão, não há a informação do tamanho da lista.

Saída

Para cada caso de teste a saída é formada por uma frase no formato "Caso de Teste *t*", onde *t* corresponde ao número de um caso de teste. Em seguida, o programa deve imprimir tantas linhas no formato "*x*/*y* equivalente a *w*/*u*" quantos forem os pares de frações equivalentes na sequência. Se não houver pares equivalentes o programa deve emitir uma linha com a mensagem: "Nao ha fracoes equivalentes na sequencia".

Exemplo

Entrada
2 1/2 1/3 1/4 1/5 1/6 1/2 2/4 3/5 10/20 3/6
Saída
Caso de teste 1 Nao ha fracoes equivalentes na sequencia Caso de teste 2 1/2 equivalente a 2/4 1/2 equivalente a 10/20 1/2 equivalente a 3/6 2/4 equivalente a 10/20 2/4 equivalente a 3/6 10/20 equivalente a 3/6

26 Frequencia do Maior e Menor

 (+++)

Dada uma matriz A de dimensões N x M contendo apenas números inteiros positivos, verifique o maior e o menor valor da matriz e conte quantas vezes estes valores aparecem.

Entrada

A entrada contem apenas em caso de teste. A primeira linha há dois números inteiros N e M, tais que 1 < N, M <= 1000, representando as dimensões da matriz A. A seguir haverão N linhas com M inteiros em cada linha separados por um espaço em branco cada, representando os elementos da matriz A. Cada elemento de A é um número inteiro tal que 0 <= a ij <= 1000;

Saída

A saída consiste de duas linhas: a primeira deve conter o menor valor da matriz A, um espaço, a porcentagem correspondente à frequência dele na matriz, com duas casas após a vírgula e em seguida o símbolo de porcentagem. A segunda linha é semelhante, contendo o maior valor de A e a porcentagem correspondente à sua frequência na matriz. Após a impressão do último valor, quebre uma linha.

Exemplo

Entrada	Saída
2 3 2 5 9 2 6 2	2 50.00% 9 16.67%

Entrada	Saída
3 3 1 2 3 2 0 4 1 0 0	0 33.33% 4 11.11%

27 Frequência de Letras

 (+++)

Neste problema estamos interessados na frequência das letras em uma dada linha de texto. Especificamente, deseja-se saber qual(is) a(s) letra(s) de maior frequência do texto, ignorando o "case sensitive", ou seja maiúsculas ou minúsculas (sendo mais claro, "letras" referem-se precisamente às 26 letras do alfabeto).

Entrada

A entrada contém vários casos de teste. A primeira linha contém um inteiro N que indica a quantidade de casos de teste. Cada caso de teste consiste de uma única linha de texto. A linha pode conter caracteres "não letras", mas é garantido que tenha ao menos uma letra e que tenha no máximo 200 caracteres no total.

Saída

Para cada caso de teste, imprima uma linha contendo a(s) letra(s) que mais ocorreu(ocorreram) no texto em minúsculas (se houver empate, imprima as letras em ordem alfabética).

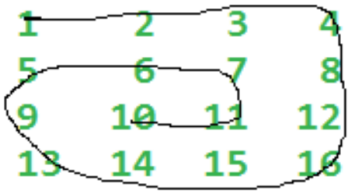
Exemplo

Entrada
3 Computers account for only 5% of the country's commercial electricity consumption. Input frequency letters
Saída
co inptu e

28 Imprime matriz em espiral (+++)

 (+++)

Faça um programa que leia uma matriz de inteiros de no máximo 10x10 elementos e a imprima de forma espiral. Por exemplo, dada a matriz abaixo, a ordem de impressão é expressa pela linha que forma uma espiral.



Entrada

A quantidade de linhas e colunas da matriz seguido dos elementos da matriz.

Saída

Uma linha contendo os elementos da matriz impressos na ordem de espiral. Caso a quantidade de linhas e colunas seja inválida, imprima a mensagem "Dimensao invalida\n"e finalize o programa.

Exemplo

Entrada	Saída
`11 -2`	`Dimensao invalida`

Entrada	Saída
`1 5` `1 2 3 4 5`	`1 2 3 4 5`

Entrada	Saída
2 2 1 2 3 4	1 2 4 3

Entrada	Saída
4 2 1 2 3 4 5 6 7 8	1 2 4 6 8 7 5 3

Entrada	Saída
4 3 1 2 3 4 5 6 7 8 9 0 1 2	1 2 3 6 9 2 1 0 7 4 5 8

Entrada	Saída
4 4 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16	1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10

Entrada	Saída
6 3 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18	1 2 3 6 9 12 15 18 17 16 13 10 7 4 5 8 11 14

29 Limpa *String* (+++)

 (+++)

Faça um programa que atualize um texto removendo uma lista de caracteres indesejados. Tanto o texto quanto a lista de caracteres devem ser lidos no formato de *strings*.

Escreva a função `str_clean` que realiza o processamento desejado. Ela deve receber como parâmetros a *string* original `str` e a *string* com caracteres indesejados `clr`. Considere o tamanho máximo de 256 caracteres.

Sua função `str_clean` deve varrer a *string* original e remover todos os caracteres que ocorrem na *string* `clr`. Use um vetor de no máximo 256 caracteres. Seu programa principal deve ser o seguinte código:

```
int main() {
   char str[N]; // string original
   char clr[N]; // lista de caracteres indesejados
   scanf("%[^\n]%*c", str);
   scanf("%[^\n]%*c", clr);
   str_clean(str, clr)  ;
   printf("%s\n", str);
   return 0;
}
```

Entrada

Seu programa deve ler duas *strings*.

Saída

Uma linha contendo a *string* modificada.

Observações

Exemplo

Entrada	Saída
Fulando de Tal da Silva aeiou	Flnd d Tl d Slv

Entrada	Saída
100 200 300 400 500 600 700 123456789	00 00 00 00 00 00 00

Entrada	Saída
1111111111x 1	x

30 Limpa *String* (+++)

 (+++)

Faça um programa que atualize um texto removendo uma lista de caracteres indesejados. Tanto o texto quanto a lista de caracteres devem ser lidos no formato de *strings*.

Escreva a função `str_clean` que realiza o processamento desejado. Ela deve receber como parâmetros a *string* original `str` e a *string* com caracteres indesejados `clr`. Considere o tamanho máximo de 256 caracteres.

Sua função `str_clean` deve varrer a *string* original e remover todos os caracteres que ocorrem na *string* `clr`. Use um vetor de no máximo 256 caracteres. Seu programa principal deve ser o seguinte código:

```c
int main() {
   char str[N]; // string original
   char clr[N]; // lista de caracteres indesejados
   scanf("%[^\n]%*c", str);
   scanf("%[^\n]%*c", clr);
   str_clean(str, clr);
   printf("%s\n", str);
   return 0;
}
```

Entrada

Seu programa deve ler duas *strings*.

Saída

Uma linha contendo a *string* modificada.

Observações

Exemplo

Entrada	Saída
`Fulando de Tal da Silva` `aeiou`	`Flnd d Tl d Slv`

Entrada	Saída
`100 200 300 400 500 600 700` `123456789`	`00 00 00 00 00 00 00`

Entrada	Saída
`1111111111x` `1`	`x`

31 Logotipo na TV da vovó

 (+++)

A Vovó está preocupada com seu netinho que passa o dia inteiro assistindo televisão, pois ela percebeu que durante o desenho animado, aparece um logotipo publicitário na tela e ela não está muito contente com isso. O netinho e a Vovó gravaram alguns programas e agora desejam verificar se o logotipo aparece durante o desenho animado. Faça um programa que, dado uma imagem codificada em números inteiros e um logotipo codificado do mesmo jeito, decida se o logotipo aparece na imagem. A matriz do logotipo é sempre de dimensões menores que as matrizes da imagem.

Entrada

A entrada possui apenas um caso de teste. O caso de teste inicia com a descrição da matriz que representa o logotipo. A primeira linha contém um numero inteiro X que representa o número de linhas e colunas da matriz do logotipo ($1 <= X <= 10$). As X linhas seguintes da entrada contém X números inteiros cada, descrevendo o valor de cada ponto da matriz do logotipo. Após a descrição do logotipo, é descrita a matriz que representa a imagem do vídeo gravado. A primeira linha da descrição possui um número inteiro Y que indica o número de linhas e colunas da matriz da imagem gravada pela Vovó ($1 <= Y <= 320$). Cada pixel é um número p com 0 p 255.

Saída

A saída consiste apenas de uma linha, contendo a palavra “sim” caso o logotipo apareça na imagem ou “nao”, caso contrario. Em ambos os casos a palavra deve ser impressa apenas com letras minúsculas e sem acentos. Após a impressão, quebre uma linha.

Exemplo

Entrada	Saída
2 1 1 2 3 5 0 0 0 2 3 0 1 1 0 0 1 3 3 0 1 3 0 0 0 2 0 0 0 1 1	nao

Entrada	Saída
2 9 9 9 1 4 2 9 9 2 3 9 1 8 8 7 9 0 9 9 2 9	sim

32 Loteria

 (+++)

A Mega-Sena é a maior loteria do Brasil. Para ganhar o prêmio máximo é necessário acertar a sena, o que significa obter coincidência entre seis dos números apostados e os seis números sorteados, de um total de sessenta dezenas (de 01 a 60), independentemente da ordem da aposta ou da ordem do sorteio. O concurso prevê também a chance de ganhar parte do prêmio, acertando a quina ou a quadra. A Mega-Sena foi lançada em março de 1996 e já premiou mais de 200 ganhadores na faixa principal. Os prêmios correspondem a 32,2% da renda das apostas ao imposto de renda correspondem 13,8% de todas as apostas. Os vencedores têm 90 dias para retirar o prêmio, se o período expirar, o dinheiro do prêmio será transferido ao Tesouro Nacional e investido em programas educacionais. Vale lembrar que a probabilidade de acerto em uma única aposta de 6 dezenas é de 1 em 50.063.860, o que representa um percentual de 0,000002%. Faça um programa que receba todas as apostas e as seis dezenas sorteadas de um concurso e mostre quantos vencedores para sena, quina e quadra houve.

Entrada

O programa receberá um inteiro N, 1 N 50000, representando a quantidade de apostas. Em seguida, em cada uma das N linhas haverá as seis dezenas de cada aposta, sendo que as dezenas estão no intervalo entre 1 e 60 e sem repetição de dezenas por apostas. Ao final das N apostas, haverá uma linha com as seis dezenas sorteadas.

Saída

A saída consiste de 3 linhas contando uma das seguintes frases: “Houve K acertador(es) da sena” ou “Houve K acertador(es) da quina” ou ainda “Houve K acertador(es) da quadra”, onde K é quantidade de acertadores para a faixa. Caso não haja acertadores a seguinte frase deve ser apresentada: “Nao houve acertador para sena” ou “Nao houve acertador para quina” ou ainda “Nao houve acertador para quadra”. Ao exibir a última frase quebre uma linha.

Exemplo

Entrada	Saída
5 23 19 8 12 60 18 14 60 12 44 54 10 8 3 12 19 33 10 33 15 7 60 12 10 22 12 19 23 33 11 23 12 33 19 10 8	Nao houve acertador para sena Houve 1 acertador(es) da quina Houve 2 acertador(es) da quadra

33 Matriz bissimétrica (+++)

 (+++)

Uma matriz bissimétrica é uma matriz quadrada que é simétrica em ambas diagonais. Faça um programa que leia uma matriz quadrada (com números inteiros) de tamanho $n \times n$, onde $1 \leq n \leq 10$, e verifique se a matriz é bissimétrica ou não.

Entrada

O programa deve ler um número inteiro n válido que representa a dimensão da matriz e, em seguida, os $n * n$ elementos da matriz.

Saída

Caso o número n esteja fora do intervalo, o programa imprime a mensagem "dimensao invalida"e encerra. Caso contrário, o programa imprime a mensagem "bissimetrica"ou "nao bissimetrica".

Observações

Exemplo

Entrada	Saída
3 1 2 3 2 5 2 3 2 1	bissimetrica

Entrada	Saída
3 1 2 3 9 5 2 3 2 1	nao bissimetrica

Entrada	Saída
21	dimensao invalida

34 Matriz Ordenada +++

 (+++)

Faça um programa que, dada uma matriz A de dimensões N x N, ordene de forma crescente as colunas da matriz.

Entrada

A entrada contém apenas um caso de teste. A primeira linha ha um inteiro N, 1 < N <= 1000, representando a dimensão das matrizes. A seguir haverão N linhas com N inteiros em cada linha separados por um espaço em branco cada, representando os elementos da matriz A.

Saída

A saída consiste de N linhas com N inteiros em cada linha separados por um espaço em branco cada, representando a matriz A após o processo de ordenação de suas colunas. Após a última linha da matriz quebre uma linha.

Exemplo

Entrada	Saída
3 1 2 11 5 10 6 7 8 9	1 2 6 5 8 9 7 10 11

Entrada	Saída
2 34 23 56 8	34 8 56 23

35 Soma e Subtração de Polinômios

 (+++)

Faça um programa para somar ou subtrair polinômios

Entrada

A primeira linha da entrada corresponde ao número de casos de teste. Cada caso de teste é formado por várias linhas. A primeira linha contém o caractere '+' ou o caractere '-', indicando se a operação é, respectivamente de soma ou de subtração de polinômios. A próxima linha contém um número inteiro n_1 que indica o número de termos do primeiro polinômio. Em seguida, há n_1 ($n_1 \leq 50$) linhas cada uma contendo um par de números c e e separados entre si por um espaço. O número c é um valor float, com sinal e corresponde ao coeficiente de um termo do polinômio. O valor e é um número inteiro positivo e corresponde ao expoente do termo do polinômio. Após o par n_1. Há um outro número inteiro n_2 ($n_2 \leq 50$) que corresponde ao número de termos do segundo polinômio, seguido, de $n-2$ linhas contendo pares de coeficientes e expoentes, como explicando para o caso do primeiro polinômio. Os pares de cada polinômio estão ordenados em ordem decrescente de expoente.

Saída

Para cada caso de teste o programa deve imprimir uma linha contendo o polinômio resultante escrito no seguinte formato:

$$sc_1X \wedge e_n sc_2X \wedge e_{n-1} \cdot sc_nX \wedge e_1$$

onde $s \in \{'+','-'\}$ é o sinal do coeficiente do polinômio, c_i é o valor do deficiente e e_i é o valor do expoente do termo i. Se durante a operação se soma ou subtração de polinômios um dos termos do polinômio resultante ficar com coeficiente igual a zero, esse termo não deve aparecer no polinômio resultante.

Sugestão

Represente cada termo do polinômio como uma struct com dois campos: *coeficiente* e *expoente*. Represente cada polinômio de entrada e também o polinômio resultante como vetores de structs. Escreva uma função para cada uma das seguintes ações:

- Leitura de um polinômio
- Impressão de um polinômio
- Soma de dois polinômios
- Subtração de dois polinômios

Exemplo

Entrada
2 + 5 -0.5 7 +4.2 4 -3 2 +1 1 -4 0 4 +3 5 +2 4 -1 2 +3 0 + 2 -5 7 -3 0 6 +2 6 +2 5 +3 4 -2.5 3 +3 2 -1.2 1
Saída
$-0.50X \wedge 7 + 3.00X \wedge 5 + 6.20X \wedge 4 - 4.00X \wedge 2 + 1.00X \wedge 1 - 1.00$ $-5.00X \wedge 7 + 2.00X \wedge 6 + 2.00X \wedge 5 + 3.00X \wedge 4 - 2.50X \wedge 3 + 3.00X \wedge 2 - 1.20X \wedge 1 - 3.00$

36 Potência de matrizes (+++)

 (+++)

Faça um programa que leira uma matriz quadrada $\mathbf{A}_{N\times N}$, sendo $0 < N <= 10$, e imprima o resultado de $\mathbf{A}^k$, onde k é uma potência inteira maior que zero. A potência k de uma matriz é dada pela operação: $\mathbf{A}^k = \prod_{i=1}^{k} \mathbf{A}$.

Entrada

O programa deve ler o valor de N, o valor de k, em seguida $N \times N$ números reais.

Saída

O programa deve imprimir a matriz resultante com precisão de 3 casas decimais.

Exemplo

Entrada	Saída
2 4 1 0 0 1	1.000 0.000 0.000 1.000

Entrada	Saída
2 2 5 8 7 1	81.000 48.000 42.000 57.000

37 Sentença Dançante

 (+++)

Uma sentença é chamada de dançante se sua primeira letra for maiúscula e cada letra subsequente for o oposto da letra anterior. Espaços devem ser ignorados ao determinar o case (minúsculo/maiúsculo) de uma letra. Por exemplo, "A b Cd"é uma sentença dançante porque a primeira letra (’A’) é maiúscula, a próxima letra (’b’) é minúscula, a próxima letra (’C’) é maiúscula, e a próxima letra (’d’) é minúscula.

Entrada

A entrada contém vários casos de teste. Cada caso de teste é composto por uma linha que contém uma sentença, que é uma string que contém entre 1 e 50 caracteres (’A’-’Z’,’a’-’z’ ou espaço ’ ’), inclusive, ou no mínimo uma letra (’A’-’Z’,’a’-’z’). A entrada termina por fim de arquivo.

Saída

Transforme a sentença de entrada em uma sentença dançante (conforme o exemplo abaixo) trocando as letras para minúscula ou maiúscula onde for necessário. Todos os espaços da sentença original deverão ser preservados, ou seja, "sentence "deverá ser convertido para "SeNtEnCe ".

Exemplo

Entrada

`This is a dancing sentence`
`  This is a dancing sentence`
`aaaaaaaaaaa`
`z`

Saída

`ThIs Is A dAnCiNg SeNtEnCe`
`  ThIs Is A dAnCiNg SeNtEnCe`
`AaAaAaAaAaA`
`Z`

38 Estiagem

 (++++) [1]

Devido às constantes estiagens que aconteceram nos últimos tempos em algumas regiões do Brasil, o governo federal criou um órgão para a avaliação do consumo destas regiões com finalidade de verificar o comportamento da população na época de racionamento. Este órgão responsável irá pegar algumas cidades (por amostragem) e verificará como está sendo o consumo de cada uma das pessoas da cidade e o consumo médio de cada cidade por habitante.

Entrada

A entrada contém vários casos de teste. A primeira linha de cada caso de teste contém um inteiro $N(1 \leq N \leq 1 \times 10^6)$, indicando a quantidade de imóveis. As N linhas contém um par de valores $X(1 \leq X \leq 10)$ e $Y(1 \leq Y \leq 200)$, indicando a quantidade de moradores de cada imóvel e o respectivo consumo total de cada imóvel (em m ^{3}). Com certeza, nenhuma residência consome mais do que 200 m^3 por mês. O final da entrada é representado pelo número zero.

Saída

Para cada entrada, deve-se apresentar a mensagem "Cidade# n:", onde n é o número da cidade seguindo a sequência (1, 2, 3, ...) e em seguida deve-se listar, por ordem ascendente de consumo, a quantidade de pessoas seguido de um hífen e o consumo destas pessoas, arredondando o valor para baixo. Na terceira linha da saída deve-se mostrar o consumo médio por pessoa da cidade, com 2 casas decimais sem arredondamento, considerando o consumo real total. Imprimir uma linha em branco entre dois casos de teste consecutivos. No fim da saída não deve haver uma linha em branco.

Exemplo

Entrada
3 3 22 2 11 3 39 5 1 25 2 20 3 31 2 40 6 65 0
Saída
Cidade# 1: 2-5 3-7 3-13 Consumo medio: 9.00 m3. Cidade# 2: 11-10 2-20 1-25 Consumo medio: 12.93 m3.

[1] Corresponde ao problema 1023 - Estiagem do Uri Online Judge -https://www.urionlinejudge.com.br/judge/pt/problems/view/1023.

39 Frequência de *Strings* (++++)

 (++++)

Escreva um programa que leia um texto e calcule a quantidade de vezes que cada *string* aparece no texto. Uma *string* é uma sequência de caracteres delimitada pelos seguintes símbolos `" .,!?()[]{}"`.

Entrada

Seu programa deve ler uma *string* que terá no máximo 2048 caracteres.

Saída

Para cada *string* presente no texto de entrada, seu programa apresentar uma linha contendo a *string* avaliada e quantidade de vezes que ela aparece no texto. A *string* deve ser apresentada entre parênteses e sua frequência é um número inteiro.

Exemplo

Entrada	Saída

`tal pai, tal o filho`	`(tal)2`
	`(pai)1`
	`(tal)2`
	`(o)1`
	`(filho)1`

Entrada	Saída

`1 2 3 4 5 5 1 2`	`(1)2`
	`(2)2`
	`(3)1`
	`(4)1`
	`(5)2`
	`(1)2`
	`(2)2`

Entrada	Saída

`Ola mundo!  Eu gosto de programacao.`	`(Ola)1`
	`(mundo)1`
	`(Eu)1`
	`(gosto)1`
	`(de)1`
	`(programacao)1`

40 Números Racionais (++++)

 (++++)

Número racional é todo o número que pode ser representado por uma razão (ou fração) entre dois números inteiros. O conjunto dos números racionais (representado por $\mathbb{Q}$) é definido por:

$$\mathbb{Q} = \{\frac{a}{b} \mid a \in \mathbb{Z}; b \in \mathbb{Z}^*\} \tag{5}$$

Em outras palavras, o conjunto dos números racionais é formado por todos os quocientes de números inteiros a e b, em que b é não nulo. O uso da letra "Q"é derivado da palavra inglesa *quotient*, cujo significado é quociente, já que a forma de escrever um número racional é o quociente de dois números inteiros. São exemplos de números racionais:

$$\frac{5}{8}; 1; 2; \frac{1}{3}; \text{-}8; -\frac{2}{5};$$

Faça um programa que defina um novo tipo de dado através de uma estrutura chamada `tRacional`, com os componentes inteiros a e b, conforme a definição anterior. Escreva as seguintes funções para operar sobre o novo tipo:

```
1
2  /**
3   * Calcula o MDC de x e y
4   * @param x
5   * @param y
6   * @return
7   */
8  int MDC(int x, int y);
9
10
11 /**
12  * Recebe dois inteiros a e b e retorna o racional
13  * @param a numerador
14  * @param b denominador
15  * @return
16  */
17 struct tRacional racional(int a, int b);
18
19 /**
20  * Recebe um racional e retorna o seu negativo (-r).
21  * @param r numero racional
22  * @return
23  */
24 struct tRacional negativo(struct tRacional r);
25
26 /**
27  * Recebe dois racionais e retorna a adição de ambos (r1 + r2).
28  * @param r1 fator esquerdo da soma
29  * @param r2 fator direito da soma
30  * @return
31  */
32 struct tRacional soma(struct tRacional r1, struct tRacional r2);
33
34 /**
35  * Recebe dois racionais e retorna o produto de ambos (r1 * r2).
36  * @param r1 primeiro fator do produto
37  * @param r2 segundo fator do produto
38  * @return
39  */
40 struct tRacional mult(struct tRacional r1, struct tRacional r2);
41
42 /**
```

```
 * Recebe dois racionais e retorna o quociente de ambos (r1/r2).
 * @param r1 numerador
 * @param r2 denominador
 * @return
 */
struct tRacional div(struct tRacional r1, struct tRacional r2);


/**
 * Recebe um racional e reduz a fração ao máximo.
 * @param r o número racional a ser reduzido
 */
void reduzFracao( struct tRacional * r);
```

Entrada

A entrada consiste de várias linhas no seguinte formato: a b operação a b, onde: $-10000 \leq a \leq 10000$, $0 < b \leq 10000$. E operação será um dos seguintes caracteres: +, -, * ou /. A entrada termina com `EOF`.

Saída

A saída consiste de várias linhas com o resultado da operação sobre os racionais. É necessário reduzir a fração ao máximo. Após a impressão do último resultado quebre uma linha.

Exemplo

Entrada	Saída
1 5 + 2 10 2 3 + 5 7 17 24 - 5 6 8 3 * 4 3 -5 2 * 4 3 8 3 / 4 3 2 3 / 2 5	2 5 29 21 -1 8 32 9 -10 3 2 1 5 3